Sidequest

Interactive Applications — Milestone 3
DS 5023

Thomas Blalock Jackson Kennedy Hudson Noyes Nathan Wan

April 16, 2026

Contents

1 Changes Since Milestone 2 **2**

2 Adaptive Interactivity **2**
2.1 Dynamic UI 2
2.2 Widget Keys 2
2.3 Callbacks 3
2.4 Dependent Dropdowns 3

3 API Integration **4**
3.1 API Selection and Setup 4
3.2 API Call Implementation 4
3.3 Error Handling 5

4 Input Validation and User Feedback **5**
4.1 Input Validation 5
4.2 User Feedback 5

5 Interactivity Self-Review **7**
5.1 Six-Criteria Evaluation 7
5.2 Usability Target Check 7
5.2.1 ISO Metrics 7
5.2.2 Five Components of Usability 7

1 Changes Since Milestone 2

The Milestone 2 concept is unchanged: Sidequest is still a proximity-based social activity app for Leo Vance, our M1 persona, who wants to spontaneously discover or host low-stakes activities without the friction of formal event platforms. The five-page architecture (Discovery Feed, Host a Quest, My Quests, Group Chat, Quest Detail) is also unchanged.

This milestone adds four new capabilities on top of that working base:

1. Adaptive interactivity (dynamic UI, keyed widgets, callbacks, and a dependent dropdown) on the Discovery Feed and Host a Quest pages.
2. A live weather API integration (OpenWeatherMap) that helps Leo decide whether outdoor quests are worth joining given current conditions.
3. Additional input validation and user-feedback patterns (spinner, toast, status, step tracker, plus expanded `st.error` / `st.warning` usage).
4. An added `activity_type` field on every quest (produced by `quest_data.py`) to support the new dependent dropdown.

2 Adaptive Interactivity

2.1 Dynamic UI

Sidequest now has two places where controls appear or disappear based on user choice. Each serves Leo's workflow directly rather than being a demonstration for its own sake.

1. "Advanced filters" toggle on the Discovery Feed (`app.py`). By default Leo sees three filters — When, Distance, and Category — matching his on-the-go, low-friction use case. Toggling `st.toggle("Advanced filters", key="advanced_mode")` reveals two additional controls: a **Host** selectbox (populated from the unique hosts in the dataset) and a **Min spots open** number input. These power-user filters let Leo target specific hosts he has met before or avoid nearly-full quests. Hiding them by default keeps the mobile, distracted use case uncluttered.

2. "Show advanced options" toggle on Host a Quest (`pages/1_Host_Quest.py`). When Leo wants to post a trivia team, he does not care about any of the extras. Toggling this reveals three optional controls: **Allow +1s** (checkbox), **Require host approval** (checkbox), and **Accessibility notes** (text input). This keeps the fast-path form short while making thoughtful hosting possible for the cases that need it.

Both patterns implement Rams' "As little design as possible" principle: the default state shows only what Leo needs in the common case, and power-user controls are one click away when needed.

2.2 Widget Keys

Every interactive widget in the app now carries a `key=` parameter. Below are three representative widgets with inline comments explaining what the key enables; all three keys are also visible in `app.py`.

- `key="filter_time"` on the When selectbox (`app.py`). The key enables the `reset_filters()` `on_click` callback to set `st.session_state["filter_time"] = "Tonight"` and have that value picked up by the widget on the next rerun. Without the key, Streamlit generates

an internal hash that the callback cannot address, so the Reset button could not clear this control.

- `key="filter_category"` on the Category selectbox (`app.py`). The key enables the `on_category_change` `on_change` callback to identify this specific widget as the parent of the dependent activity-type dropdown. When the callback fires, it clears `filter_subtype` before the script reruns, ensuring the sub-type never holds a stale value that is invalid for the newly selected category.
- `key=f"join_{quest['id']}"` on the Accept Quest button inside the card loop (`app.py`). Because buttons are rendered inside a `for` loop, Streamlit needs a unique identifier per iteration — otherwise every card's button would share the same identity and Streamlit would raise a duplicate-widget exception. Interpolating the quest id into the key guarantees uniqueness across all rendered cards.

All other interactive widgets (`filter_distance`, `filter_subtype`, `advanced_mode`, `filter_host`, `filter_min_spots`, `weather_city`, `chart_dist`, `scatter_cats`, `host_title`, `host_category`, `host_location`, `host_date`, `host_time`, `host_group_size`, `host_desc`, `host_advanced`, `host_plus_ones`, `host_approval`, `host_accessibility`, `host_preview`, `host_publish`, `my_quests_view`, `chat_selector`, the per-chat `chat_input_<id>` / `send_<id>` pair, and `quest_picker`) also carry keys for the same reasons: to address them from callbacks, to disambiguate them in loops, and to preserve state across reruns.

2.3 Callbacks

Sidequest uses one `on_click` callback and one `on_change` callback. Each solves a problem that cannot be cleanly expressed with an `if` statement.

`on_click` — `reset_filters()` (`app.py`). A single Reset Filters button clears six different widgets at once: When, Distance, Category, Activity Type, Advanced mode, Host, and Min spots. Because Streamlit widgets read their default from `st.session_state` on each rerun, the callback writes the defaults into `st.session_state` *before* the script re-executes. A plain `if st.button(...)` block would run *after* the widgets have already been constructed for this rerun, so the state changes would not take effect until the following interaction — which would feel like a "lag" to the user. The callback pattern makes the reset take effect immediately on the same frame.

`on_change` — `on_category_change()` (`app.py`). When the Category selectbox changes value, this callback sets `st.session_state["filter_subtype"] = "All"`. This clears the dependent activity-type dropdown so that its previously-selected value (e.g., "Board Games" while Category was Games) does not persist and produce an invalid state when Category switches to Sports. Like the reset case, placing this logic in a callback guarantees the cleanup runs *before* the activity-type dropdown is rebuilt with its new options.

2.4 Dependent Dropdowns

The Discovery Feed has a pair of dependent dropdowns: **Category → Activity Type**.

The parent is the Category selectbox in the top filter row. The dependent dropdown sits directly below the filter row and is labeled "Activity type." Its options are computed from `quest_data.SUBCATEGORIES` on every rerun:

- When Category is **Sports**, Activity type offers: All, Volleyball, Basketball, Frisbee, Soccer, Running.

- When Category is **Games**, Activity type offers: All, Board Games, Card Games, Video Games, Tabletop RPG.
- When Category is **All Types**, Activity type is disabled and only shows "All."

The options visibly change as soon as the user picks a new category. This relationship is meaningful for Leo: within the broad category of Sports he very much cares *which* sport — volleyball is not basketball. A single flat category list would either be too coarse ("Sports" is the only Sports option) or too crowded (putting all 26 sub-types into one dropdown). The two-level parent/child structure keeps the surface simple while giving Leo the precision he needs.

3 API Integration

3.1 API Selection and Setup

API	OpenWeatherMap "Current Weather Data"
Base URL	`https://api.openweathermap.org/data/2.5/weather`
Authentication	API key passed as the `appid` query parameter
Rate limits	60 calls/minute, 1,000 calls/day on the free tier
Cost	Free tier, no expiration (covers the full semester and beyond)

Table 1: API selection summary.

Why this API is valuable for Leo. Leo's M1 pain point is showing up to an activity that "doesn't work out" — including outdoor activities ruined by weather. Surfacing current conditions at the top of the feed lets him decide *before* tapping Accept whether a hike or pickup volleyball is actually a good idea right now.

Key storage. The key is read from an `OPENWEATHER_API_KEY` environment variable loaded from a `.env` file via `python-dotenv`. The `.env` file is listed in `.gitignore` so the key never reaches the repository. No key string ever appears in any `.py` file.

3.2 API Call Implementation

The API call lives in `fetch_weather(city)` in `app.py` and is driven by user input: a `st.text_input` with `key="weather_city"` inside an expandable " Current Weather" section. Whatever Leo types becomes the `q` query parameter. There is no hardcoded location — he can check the weather at the quest's city, at his destination, or anywhere else.

The function builds the request with `requests.get(url, params=..., timeout=5)`, checks the status code, parses the JSON response, and returns either the raw data (on 200 OK with a valid body) or a tagged error dict. The display layer then selects the appropriate Streamlit component: four `st.metric` tiles (Temp, Feels Like, Humidity, Wind) on success, an `st.warning` for soft failures (empty results / 404), an `st.error` for hard failures (auth, rate limit, server, timeout), and a conditional `st.warning` about outdoor conditions when the weather code is rain, drizzle, thunderstorm, or snow.

Caching. `fetch_weather` is decorated with `@st.cache_data(ttl=600)`. A 10-minute TTL was chosen because (a) weather conditions do not change meaningfully within 10 minutes, (b) the free OpenWeatherMap tier allows only 1,000 calls per day, and a page that refreshes on every widget interaction would exhaust that quickly without caching, and (c) 10 minutes is short enough that

a user who sits with the app open for an hour still gets a fresh reading. An explanatory code comment accompanies the decorator.

3.3 Error Handling

The `try/except` block in `fetch_weather` covers six scenarios, exceeding the required four:

Scenario	HTTP / exception	Message shown to the user
Invalid API key	401 Unauthorized	"API key is missing or invalid." (`st.error`)
Resource not found	404 Not Found	"No results found for your search." (`st.warning`)
Rate limit exceeded	429 Too Many Requests	"API limit reached. Please wait a minute and try again." (`st.error`)
Server error	500+	"The service is temporarily unavailable. Please try again later." (`st.error`)
Network timeout	`requests.Timeout`	"Could not connect. Check your internet connection." (`st.error`)
Empty results	200 OK, empty body	"Your search returned no results. Try broader terms." (`st.warning`)

Table 2: Error scenarios handled by the weather API integration.

Each branch returns a tagged dict of the form {`"error": "...", "message": "..."`}, and the display layer chooses between `st.error` (hard failures) and `st.warning` (recoverable / empty). On the success path, `st.success("Weather loaded for <city>!")` is shown before the metric tiles, giving Leo an unambiguous positive confirmation that the call succeeded.

4 Input Validation and User Feedback

4.1 Input Validation

The app validates six scenarios across two pages, exceeding the required four. Each validation uses a clear message and prevents the app from crashing or showing an empty/broken output. Fatal validations are followed by `st.stop()`.

The multiselect case is the clearest use of `st.stop()`: if Leo deselects all categories, Plotly would receive an empty dataframe and render a confusing empty chart. Stopping the script after the warning cleanly prevents the broken output.

On the Host a Quest page, the title-length and past-time checks render inline *before* the Publish button, so Leo sees the problem while he is still typing and has a chance to fix it. The required-field checks inside the Publish button handler use `st.stop()` to guarantee that a malformed quest can never be written to session state, no matter what state the form is in.

4.2 User Feedback

Sidequest implements five feedback patterns, exceeding the required four:

Scenario	Message	Mechanism
Empty multiselect on scatter chart	"Pick at least one category to display the chart."	`st.warning` + `st.stop()`
Missing Quest Title on Publish	"**Title** is required."	`st.error` + `st.stop()`
Missing Location on Publish	"**Location** is required."	`st.error` + `st.stop()`
Quest title under 3 characters	"Quest title must be at least 3 characters."	Inline `st.error`
Start time in the past	"Start time cannot be in the past."	Inline `st.error`
Empty chat message	"Type a message before sending."	`st.warning`

Table 3: Input validation scenarios.

Pattern	Where used	File
`st.spinner`	Wrapping the weather API call while it awaits the HTTP response	`app.py`
`st.toast`	"Filters cleared!" fired inside the `reset_filters` callback	`app.py`
`st.success` / `error` / `warning` / `info`	Weather status, quest publish success, validation failures, no-match filters, empty chat lists, "You left this quest" info, "Quest is full" error	throughout all pages
`st.status`	Multi-step publish flow on Host a Quest: "Validating..." → "Creating chat..." → "Publishing..." → "Quest published!" (expanded, then `state="complete"`)	`pages/1_Host_Quest.py`
Step tracker	" Step 1 of 2 — Fill in quest details" and " Step 2 of 2 — Review and publish" banners bracket the host flow	`pages/1_Host_Quest.py`

Table 4: User feedback patterns.

5 Interactivity Self-Review

5.1 Six-Criteria Evaluation

The table on the following pages evaluates Sidequest across the six criteria from Module 5.

5.2 Usability Target Check

5.2.1 ISO Metrics

Effectiveness. Our M1 target was that a user can locate a nearby activity, tap "Accept Quest," and reach the group chat; and separately, publish a new quest with title/location/time. Both task flows work end-to-end in the current app. Evidence: a tester can open the Discovery Feed, filter to Sports, tap Accept Quest, and see the quest appear in My Quests and in Group Chat with a seeded welcome message. Hosting a quest produces a new feed entry, a new My Quests row, and a chat. We have not run structured usability sessions yet, so this remains self-evidence rather than participant-verified evidence.

Efficiency. M1 target was *join a quest in under 60 seconds / 3 taps* and *host a quest in under 90 seconds*. The join path today is 3 taps (open feed → optionally filter → Accept Quest), and with the Advanced filters hidden by default no extra clicks are added. The host path is slightly heavier than the M1 target because we kept Preview as a separate step before Publish, which is a conscious trade-off favoring error prevention (Shneiderman reversal) over raw speed. Bottleneck: the host page has enough fields that even a fast typist will land near 60 seconds, not well under 45.

Satisfaction. M1 target was a Single Ease Question score of 4/5 or better. We have not run participant testing yet, but in internal team testing the green accent, rounded buttons, and `st.balloons` on publish/join produce the intended "fun" feel. Likely complaint: the app still looks like a Streamlit app in ways that are hard to hide (sidebar chrome, the "Deploy" button in dev, etc.), which is inherent to the framework.

5.2.2 Five Components of Usability

- **Learnability.** The Discovery Feed's three-filter layout is readable without instruction: the top row is filters, the middle is results, the bottom is stats and charts. The Advanced toggle starts off, so a first-time user sees only the simple version.
- **Efficiency.** The Reset Filters button (`on_click=reset_filters`) clears six controls in one tap. The Advanced filters toggle avoids hiding the simple case behind an accordion for normal users. Cache decorators keep reruns fast.
- **Memorability.** Navigation is the Streamlit sidebar, which is always visible. Page names are task-oriented ("Host a Quest," "My Quests," "Group Chat") rather than object-oriented, which is easier to re-learn after a gap.
- **Errors.** Every fatal validation path ends in `st.stop()`, preventing malformed state from being written. Inline title-length and past-time validation catches mistakes while typing rather than at submit time.
- **Satisfaction.** Positive reinforcement is layered: `st.balloons` on successful publish, `st.toast` on filter reset, `st.success` on weather load, and `st.status` with a green check on publish complete. These small celebratory moments match the "exploration rather than administrative work" brand tone from M1.

Criteria	Score	Evidence	What could improve
Visualization	4	Two Plotly charts on the Discovery Feed: a category bar chart with a distance slider, and a distance-vs-time scatter with a category multiselect and spot-size encoding. Both charts use hover tooltips to surface quest metadata (`app.py`, bar at the `fig_bar` block, scatter at `fig_scatter`).	Add a map visualization showing quest locations geographically, so Leo can see proximity as a spatial layout rather than a numeric column.
Layout	4	Consistent 700px max-width, three-column filter row, bordered cards, sidebar navigation, and a unified green primary-button style across all five pages. Applied via a shared CSS block injected at the top of every page.	Move page-level CSS into a single `style.css` loaded from one place rather than duplicating the block in every file. Reduces drift risk as the app grows.
Basic Interactivity	5	Every input widget meaningfully drives output: filters narrow the quest list, the chart slider resizes the bar chart, the chart multiselect subsets the scatter plot, the date/time/slider controls on Host produce real quests, and the "Accept Quest" button writes to `st.session_state` and re-renders card states.	None critical. Could add a "Join n at once" batch button for the power case of joining several trivia-team-sized quests, but that is a feature request, not a defect.
Adaptive Interactivity	4	Dynamic UI in two places (Advanced filters toggle, Advanced options toggle), two callbacks (`reset_filters`, `on_category_change`), dependent Category → Activity Type dropdown, and every widget carries a key. See `app.py` and `pages/1_Host_Quest.py`.	Add a second dependent dropdown (e.g., a neighborhood → specific venue pair) so the pattern feels like a general capability of the app rather than a one-off on Category.

Criteria	Score	Evidence	What could improve
Performance	4	@st.cache_data on load_quests() and load_quest_db() avoids regenerating mock data on every rerun. @st.cache_data(ttl=600) on fetch_weather prevents rate-limit exhaustion.	When the dataset grows, iterrows() on the filtered dataframe (used in the card loop) will become slow. Switching to a vectorized or component-based card render would scale better.
Feedback & Validation	4	Five feedback patterns implemented (spinner, toast, success/error/warning/info, status, step tracker). Six validation scenarios across two pages with st.stop() on fatal paths and inline validation for title-length and past-time checks while typing.	Add inline character-count feedback on the Quest Title field so Leo sees remaining characters before hitting the 3-character floor. Consolidate validation logic into a shared validate_quest() helper to reduce duplication across pages.